Session 5 · Agents, Governance, Your Capstone

Until now you wrote every plan. Now the model plans its own steps, within limits you define.

Plan: concepts 10' · demo 18' · lab 30' · course wrap-up 12' · close 5'

By the end of Session 5 you can

1. **Explain** what separates an agent from a workflow, and when each wins
2. **Read** an agent loop and point at every governance lever in the code
3. **Run** a governed agent on your own company pair, and pass its human gate
4. **Publish** your finance tool on GitHub, with a README that claims only what the tool does

What we are doing, and why

- **An agent is a model granted autonomy**: it chooses which tool to call next, in a loop, until it judges the goal met — or reaches limits you set.
- **Autonomy is granted like a mandate**: explicit rules, hard limits, a human signature at the end. You write all three today, in code.
- **The whole mechanism is about forty lines**, and you will read every one of them before granting the model any autonomy.
- **The industry frameworks come last**: PydanticAI runs the same discipline with your own tool, and a mapping table places every lever in LangGraph, whose workflow half you built in Session 4.

The storyline of this session

```
the agent's anatomy              forty lines, every governance lever visible
        ↓
Apple vs Sony                    the task — and Sony files in Japanese yen
        ↓
the currency check, in code      your tool detects mixed units and raises a warning
        ↓
your rules govern the run        the agent plans, fetches, compares — governed
        ↓
the same agent in PydanticAI     your tool, unchanged, in an industry framework
        ↓
publish on GitHub                the finished tool, public under your name
```

Workflow vs agent

	Workflow (Session 4)	Agent (now)
The plan	fixed, written by you	**chosen by the model**, step by step
Tools	your code calls them	the model requests, **your code executes**
Stops when	the script ends	when it decides, or at **the limits you set**
Use for	repeatable work	paths that **depend on the data**

The agent mechanism is a loop

```
while steps < MAX_STEPS:                    # the safety limit — never the goal
    response = claude(messages, tools=TOOLS)
    if response.wants_tool:
        result = run_tool(...)              # your code executes
        messages += [response, result]      # the conversation is the memory
    else:
        break                               # the model judges the goal met
```

About forty lines. Once you have read them, you can evaluate any vendor's agent claims from first principles — the notebook then runs the same discipline in **PydanticAI** and maps every lever to **LangGraph**.

Governance: six levers, all in code

Lever	Failure it prevents
MAX_STEPS	the agent looping indefinitely
TOKEN_BUDGET	unbounded spend
Tool whitelist	doing anything you gave no tool for
Forced output schema	an essay instead of a decision
Human gate	anything saved without explicit approval
Audit trail	decisions that cannot be reconstructed afterwards

Tools give actions; skills give knowledge

	Tool	Skill
What it is	a function the agent may call	a file of know-how the agent may load
It carries	an action: fetch, compute, compare	a procedure: house style, checklist, method
Advertised by	name + description + schema	name + one-line description
When chosen	your code executes it	its full text enters the context
In your lab	`compare_metrics`, `forecast_revenue`	`memo-style` — the desk's memo rules

The demonstration: a currency trap

Task: compare Apple with Sony — its oldest consumer-electronics rival.

```
naive agent:  JPY 13tn  vs  USD 416bn  →  "Sony is 30x bigger"       (wrong)
our agent:    tool detects mixed units → warning → compares
              growth and margins only                              (correct)
```

- **Defense in depth**: a rule in the prompt *and* a check in the tool
- You build both protections yourself, then run your own pair

The libraries in this session

Library	Role here	Standing
`pydantic`	the typed recommendation	the industry standard for validation
PydanticAI	the agent loop, packaged — runs live, using your own tool	a leading agent framework, from the Pydantic team
LangGraph	agents as graphs (you built its workflow half in Session 4)	the other leading agent framework

How the labs work

Every exercise sits between two markers. **You fill the gaps. Nothing else changes.**

```
### START CODE HERE ###
"3. ALWAYS check the [WHICH FIELD?] field before comparing figures."
### END CODE HERE ###
```

- `None` → replace with the correct column, value or variable
- `[QUESTION IN CAPITALS]` → replace with the text the bracket asks for
- **Everything else is given.** Do not rewrite it.
- Then run the ✅ **check cell** directly below. Green means correct: continue.

Stuck for two minutes? Select the lines, press `Option+K` (`Alt+K` on Windows), and ask the ✱ panel.

Your lab, then your capstone submission

Lab (30'): build the currency tool + the agent's rules → run your own pair → pass the gate → **publish your repository with its README**

Stretch: a second real tool — your revenue forecaster — then a skill: the desk's memo style, loaded on demand

Capstone — a submission, not a presentation: your repository URL + a one-page memo inside the repository:

1. The **job** your workflow does
2. A **real run** on your own company pair
3. The **trust story**: one failure you caught + the check that caught it
4. One honest **number**
5. The **next step**

What you own now — public, under your name

One tool, on your GitHub:

- **Grounded prompts** that refuse rather than guess
- **A comparable-company valuation** of Apple, from live SEC data
- **Sanity checks and an evidence verifier** for everything a model writes
- **A screening workflow** on live filings, in LangGraph
- **A governed agent**, with every limit in code

A challenge for the coming week: choose one recurring task at your desk, build it as a workflow, and show a colleague.

Never ship a number you haven't verified.